

Pimcore Customer Data Object Implementation & Troubleshooting Report

Project Overview

Task

Create a Customer Information Data Object in Pimcore to store and manage customer-related information in a structured and reusable format.

Objective

The goal was to:

- Understand Pimcore Data Objects
 - Design a scalable Customer data model
 - Configure appropriate field types
 - Create and manage Customer records
 - Verify data persistence
 - Understand Pimcore Studio, MariaDB, OpenSearch, and Docker integration
-

Approach Followed

Before directly implementing the task, a systematic troubleshooting and verification approach was used.

The approach consisted of:

1. Environment Verification
2. Pimcore Backend Validation
3. Database Validation
4. OpenSearch Validation
5. Class Design
6. Data Object Modeling
7. Object Creation
8. Publishing & Verification
9. Database Validation of Final Results

The idea was to first ensure the platform was functioning correctly before creating business objects.

Phase 1 – Initial Issues Encountered

Issue 1: Pimcore Admin Panel Not Accessible

Problem

Accessing:

```
http://localhost:8082/admin
```

produced:

```
404 Not Found  
No route found for "GET /admin"
```

Investigation

Initially it appeared that Pimcore Admin was broken.

To determine whether Pimcore itself was broken or only the route was unavailable, multiple checks were performed.

Finding

The project was configured to use:

```
Pimcore Studio
```

instead of the legacy Pimcore Admin interface.

Correct URL:

```
http://localhost:8082/pimcore-studio
```

Resolution

Used Pimcore Studio for all further work.

Issue 2: Verifying Whether Pimcore Documents Were Loading Correctly

Problem

Needed to verify whether Pimcore was able to load Document objects.

Approach

Executed a PHP script inside the PHP container.

Command:

```
docker exec -it my-pimcore-new-php-1 php -r '
require "vendor/autoload.php";

$kernel = new App\Kernel("dev", true);
$kernel->boot();

$d = \Pimcore\Model\Document::getById(1);

var_dump($d);
'
```

Purpose

To confirm:

- Pimcore bootstrap works
- Database connectivity works
- Documents can be loaded

Finding

The expected output was not returned.

Further investigation was required.

Issue 3: Database Validation

Problem

Needed to determine whether the issue was in Pimcore or the database.

Approach

Connected directly to MariaDB.

Command:

```
docker exec -it my-pimcore-new-db-1 mysql -upimcore -ppimcore pimcore
```

Checked available document tables.

```
SHOW TABLES LIKE 'documents_page';
```

Verified root document.

```
SELECT * FROM documents_page WHERE id = 1;
```

Checked editable content.

```
SHOW TABLES LIKE 'documents_editables';  
  
SELECT COUNT(*) FROM documents_editables;
```

Findings

- Document tables existed.
- Root document existed.
- Editable table was empty.

Conclusion

Database was healthy.

The issue was not related to missing document records.

Issue 4: Pimcore Studio API Validation

Objective

Verify whether Pimcore API could fetch documents.

Approach

Accessed:

```
http://localhost:8082/pimcore-studio/api/documents/1
```

Findings

JSON response returned successfully.

Conclusion

Confirmed:

- Pimcore backend was running.
- API endpoints were functional.
- Database records were accessible.

Phase 2 – Customer Data Model Design

After verifying system stability, implementation of the Customer Information Data Object started.

Requirement Analysis

The Customer model needed to store:

- First Name
- Last Name
- Email
- Phone Number
- Date Of Birth
- Gender
- Address
- Customer Status

The design had to be:

- Structured
- Reusable
- Maintainable
- Extensible

Class Design

Created a Pimcore Data Object Class:

Customer

Why a Data Object?

Data Objects are ideal for:

- Structured business data
- Relationships
- Validation
- Searchability
- Future scalability

Field Configuration

First Name

Field Type:

Input

Reason:

Single-line textual value.

Last Name

Field Type:

Input

Reason:

Simple textual value.

Email

Field Type:

Input

Reason:

Stores email string.

Phone Number

Field Type:

Input

Reason:

Phone numbers may contain:

- Country codes
- Spaces
- Symbols

Therefore Input field was preferred over Numeric.

Date Of Birth

Field Type:

Date

Reason:

Supports date picker and proper storage.

Gender

Field Type:

Select

Values:

Male
Female
Other

Reason:

Restricts input to predefined values.

Address

Field Type:

Textarea

Reason:

Supports multiline content.

Customer Status

Field Type:

Select

Values:

Active
Inactive
Blocked

Reason:

Ensures data consistency.

Key Design Decisions

Mandatory Fields

Marked as mandatory:

- First Name
- Last Name
- Email

Reason:

Core customer identification data.

Select Fields

Used for:

- Gender
- Status

Reason:

Avoids inconsistent user input.

Example:

Male
male
MALE

would otherwise create inconsistent data.

Phase 3 – Folder Creation

Created folder:

/customers

inside the Data Object Tree.

Purpose:

- Organize Customer objects
- Maintain clean hierarchy
- Improve scalability

Issue 5: Folder Already Exists

Error

Folder already exists

Investigation

Reviewed Data Object Tree.

Found:

/customers

already existed.

Resolution

Used the existing folder.

Phase 4 – OpenSearch Indexing Issue

Issue 6: Customer Object Creation Failed

Error

```
Failed to save new element  
  
no such index  
pimcore_data-object_customer
```

Impact

Customer objects could not be created.

Troubleshooting Process

Step 1

Entered OpenSearch container.

```
docker exec -it my-pimcore-new-opensearch-1 bash
```

Step 2

Checked indexes.

```
curl -k -u 'admin:<password>'  
'https://localhost:9200/_cat/indices?v'
```

Finding

Customer index existed.

```
pimcore_data-object_customer-even
```

Step 3

Checked aliases.

```
curl -k -u 'admin:<password>'
'https://localhost:9200/_cat/aliases?v'
```

Finding

Alias existed correctly.

```
pimcore_data-object_customer
→ pimcore_data-object_customer-even
```

Root Cause Analysis

The error initially suggested:

```
missing index
```

However investigation showed:

- Index existed
- Alias existed
- OpenSearch cluster healthy

Therefore the issue was likely caused by:

- temporary synchronization delay
- stale Studio state
- indexing refresh timing

Resolution

Refreshed Studio and retried object creation.

Object creation succeeded.

Phase 5 – Customer Object Creation

Created folder structure:

/customers

Created Customer object:

Jon Snow

Customer Data Entered

First Name : Jon
Last Name : Snow
Email : jonsnow@gmail.com
Phone Number : 987654321
Date Of Birth : 02/06/2026
Gender : Male
Address : Winterfell
Customer Status : Active

Publishing Strategy

Used:

Save & Publish

instead of:

Save Draft

Reason:

Published objects become available to listings, search indexes, and production workflows.

Verification Phase

Database Verification

Connected to MariaDB.

```
docker exec -it my-pimcore-new-db-1 mysql -upimcore -ppimcore pimcore
```

Executed:

```
SELECT id,parentId,`key`,published  
FROM objects  
ORDER BY id;
```

Results

```
ID 6  
Jon Snow  
published = 1
```

Interpretation

Successfully confirmed:

- Object exists
- Object saved
- Object published
- Data persisted

Key Findings

Finding 1

Pimcore Studio replaces the legacy Admin UI in this setup.

Finding 2

Database integrity should always be verified before application-level debugging.

Finding 3

OpenSearch errors do not always indicate missing indexes.

Aliases and cluster health should also be checked.

Finding 4

Publishing state affects object visibility.

Objects saved as Draft:

```
published = 0
```

may not appear in certain listings.

Finding 5

Data Objects provide a highly scalable approach for business entities like Customers.

Additional fields can be added later without redesigning the structure.

Tools Used

Pimcore Studio

Used for:

- Class creation
 - Folder creation
 - Customer object creation
 - Publishing
-

Docker

Used for:

- Container access
- Service diagnostics

Commands:

```
docker exec -it my-pimcore-new-php-1 bash

docker exec -it my-pimcore-new-db-1 mysql -upimcore -ppimcore pimcore

docker exec -it my-pimcore-new-opensearch-1 bash
```

MariaDB

Used for:

- Object verification
- Database inspection

OpenSearch

Used for:

- Index verification
- Alias verification
- Search troubleshooting

Symfony

Used indirectly through Pimcore runtime and console commands.

Final Outcome

Successfully implemented a complete Customer Information Data Object system in Pimcore.

Achievements:

- ✓ Resolved environment and routing issues
- ✓ Verified database integrity
- ✓ Verified OpenSearch configuration
- ✓ Designed Customer Data Model
- ✓ Configured all required fields
- ✓ Created Data Object folder hierarchy
- ✓ Created Customer records
- ✓ Published Customer objects
- ✓ Validated database persistence
- ✓ Confirmed successful end-to-end functionality

The solution is reusable, maintainable, scalable, and ready for future enhancements.